

## Parte A — De onde veio esse problema

Na aula passada (slide 15) vimos por exemplo que, para um filtro de média  $k \times k$ , cada pixel de saída é a média de  $k^2$  amostras independentes do ruído gaussiano. Pela lei do  $\sqrt{N}$ , o desvio-padrão do ruído cai por um fator  $\sqrt{k^2} = k$ :

- kernel  $3 \times 3 \rightarrow 9$  vizinhos  $\rightarrow$  ruído cai por  $3\times$
- kernel  $5 \times 5 \rightarrow 25$  vizinhos  $\rightarrow$  ruído cai por  $5\times$
- kernel  $7 \times 7 \rightarrow 49$  vizinhos  $\rightarrow$  ruído cai por  $7\times$

Como o SNR é  $\mu_f/\sigma_\eta$ , reduzir  $\sigma_\eta$  pelo fator  $k$  multiplica o SNR por  $k$ . Quanto maior o kernel, maior o ganho.

Tudo bem — só que tem um problema escondido nesse raciocínio: **para escolher  $k$ , você precisa saber quanto vale  $\sigma_\eta$** . Se o ruído for muito fraco, kernel grande borra a imagem sem necessidade. Se for muito forte, kernel pequeno deixa ruído passar. A escolha certa depende do parâmetro  $\sigma$ .

Na vida real, tal como disse na aula, você raramente recebe essa informação. Você baixa uma imagem médica anonimizada, captura uma foto com a câmera de outra pessoa, recebe um frame de um sensor cujo datasheet você não tem. E mesmo quando você *tem* o sensor em mãos, o ISP da câmera precisa estimar  $\sigma$  **em tempo real**, frame a frame, para decidir a força do filtro de denoising que vai aplicar antes de te entregar a foto. Sem essa estimativa, ele não sabe se está filtrando demais (perde detalhe) ou de menos (deixa ruído).

A pergunta deste laboratório é direta:

*Como medir o ruído de uma imagem sem nunca ter visto a imagem limpa?*

## Imagem de trabalho

Vamos usar a mesma imagem da Figura 5.7 do Gonzalez (3ª edição) — a radiografia de uma placa de circuito eletrônico, em 8 bits. Essa imagem foi escolhida por dois motivos: tem regiões **relativamente lisas** (o substrato da placa) intercaladas com **bordas fortíssimas** (trilhas, pinos do encapsulamento, contornos dos componentes). É o cenário ideal para esticar os estimadores que vamos construir.

No livro, o exemplo 5.2 trabalha com ruído gaussiano de **média zero e variância 400** sobre essa imagem em 8 bits. Convertendo:  $\sigma = \sqrt{400} = 20$  em escala  $[0, 255]$ , ou seja  $\sigma \approx 0,078$  em escala normalizada  $[0, 1]$ . Esse é o caso “ruído significativo” segundo os autores — vai ser o nosso ponto central de comparação.

*O livro cita variância 400 como valor de referência, mas se você comparar pixel a pixel a Fig 5.7(a) com a Fig 5.7(b) disponível no site oficial, vai ver que o ruído real adicionado tem  $\sigma \approx 27$  e média  $\approx -9$  — provavelmente fruto de saturação e arredondamento na produção do livro. Por isso, no nosso laboratório vamos gerar nosso próprio ruído sobre a Fig 5.7(a), de modo a conhecer com precisão o  $\sigma$  que estamos tentando estimar.*

## Parte B — A primeira tentativa (e o tombo)

### Atividade B.1

Carregue a imagem `Fig0507_a_ckt-board-orig.tif` e adicione ruído gaussiano com  $\sigma = 0,08$  — equivalente à variância 400 do exemplo do Gonzalez. Calcule `np.std` da imagem ruidosa.

```
1 import numpy as np
2 from PIL import Image
3
4 rng = np.random.default_rng(42)
5
6 f = np.array(Image.open(
7     'Fig0507_a_ckt-board-orig.tif'),
8     dtype=float) / 255.0
9
10 sigma_verdadeiro = 0.08
11 ruído = rng.normal(0, sigma_verdadeiro, f.shape)
12 g = np.clip(f + ruído, 0, 1)
13
14 print(f"sigma (global) = {g.std():.4f}")
15 print(f"sigma verdade = {sigma_verdadeiro:.4f}")
```

**Pergunta:** o valor estimado bate com o verdadeiro? Por quanto erra?

### Atividade B.2

Agora repita `np.std`, mas **na imagem limpa**:

```
1 print(f"std da imagem limpa = {f.std():.4f}")
```

Se `np.std` fosse um bom estimador de  $\sigma$ , deveria dar quase zero numa imagem sem ruído. Não dá: dá em torno de 0,26. Por quê?

## Explicação: variância espacial vs. variância do ruído

Quando você calcula `np.std(g)`, está medindo a variabilidade dos valores dos pixels **no espaço da imagem** — o quanto eles diferem entre si. Essa variação tem duas fontes:

1. **Estrutura do sinal:** as trilhas são escuras, o substrato é claro, os pinos têm alto contraste, e há ainda o fundo preto fora da placa. Mesmo sem ruído nenhum, os pixels variam bastante entre si — eles *deveriam* variar, é o que torna a imagem reconhecível.
2. **Ruído aleatório:** a flutuação que se sobrepõe à imagem.

Como sinal e ruído são independentes, a variância da soma é a soma das variâncias:

$$\text{Var}_{\text{esp}}(g) = \text{Var}_{\text{esp}}(f) + \text{Var}(\eta) \quad (1)$$

Para a placa de circuito,  $\sigma_f \approx 0,26$  e  $\sigma_\eta = 0,08$ . Substituindo:

$$\sigma_g = \sqrt{0,26^2 + 0,08^2} \approx 0,272$$

O ruído está lá, mas perdido dentro da variabilidade do sinal. A estrutura da imagem domina a estimativa, e  $\sigma_f$  não tem nada a ver com  $\sigma_\eta$ .

**Conclusão:** para medir só o ruído, precisamos de uma estatística que **ignore a estrutura** da imagem e olhe apenas para a flutuação aleatória.

## Parte C — Duas técnicas que funcionam

### C.1 — Diferenças entre pixels vizinhos

A ideia central: em regiões **suaves** da imagem (por exemplo, no substrato da placa, longe das trilhas), pixels vizinhos têm valores parecidos. Se subtrairmos um do outro:

$$d[i] = g[i + 1] - g[i]$$

Expandindo  $g = f + \eta$ :

$$d[i] = \underbrace{(f[i + 1] - f[i])}_{\approx 0 \text{ em reg. suave}} + \underbrace{(\eta[i + 1] - \eta[i])}_{\text{só ruído}}$$

O sinal se cancela, e sobra a diferença de dois ruídos. Cada ruído é gaussiano com variância  $\sigma^2$ ; a diferença de dois gaussianos independentes é também gaussiana, mas com variância  $2\sigma^2$  (variâncias se somam mesmo na subtração, porque o sinal não importa para variância). Logo:

$$\text{std}(d) = \sqrt{2}\sigma$$

$$\hat{\sigma}_{\text{diff}} = \frac{\text{std}(d)}{\sqrt{2}} \quad (2)$$

A divisão por  $\sqrt{2}$  **não é arbitrária** — ela compensa o fato de que estamos medindo a diferença entre dois ruídos em vez de um só.

### Atividade C.1

Implemente e teste o estimador:

```
1 def estimar_sigma_diff(img):
2     d = np.diff(img, axis=1)
3     return d.std() / np.sqrt(2)
4
5 print(f"sigma verdadeiro: {sigma_verdadeiro:.4f}")
6 print(f"sigma (diff): {estimar_sigma_diff(g):.4f}")
```

**Pergunta:** agora o erro é grande ou pequeno? Compare com o resultado da Parte B.

## Onde o método das diferenças falha

A hipótese “vizinhos têm valores parecidos” vale na maior parte da imagem, mas **quebra em bordas reais**. Olhe para a região superior da placa, onde estão os pinos do conector — as transições do escuro do pino para o claro do substrato são abruptas. Nessas transições,  $f[i + 1] - f[i]$  pode valer 0,5 ou mais. Isso vira um valor enorme na lista de diferenças e infla o desvio-padrão.

### Atividade C.2

Recorte a região superior da placa (muitos pinos verticais) e estime nela com o mesmo  $\sigma = 0,08$ :

```
1 recorte_pinos = g[0:120, :]
2
3 print(f"diff (inteira): "
4       f"{estimar_sigma_diff(g):.4f}")
5 print(f"diff (pinos): "
6       f"{estimar_sigma_diff(recorte_pinos):.4f}")
```

**Observação:** o estimador superestima em regiões com bordas. Quanto mais bordas, mais ele erra para cima.

### C.2 — MAD: ignorando as bordas com elegância

Para consertar isso, trocamos o desvio-padrão por uma estatística que **não é afetada por valores extremos**: o MAD (*median absolute deviation*, ou mediana dos desvios absolutos).

**Como o MAD funciona:** dado um vetor, calcula-se primeiro a mediana, depois a mediana dos módulos dos desvios em relação a ela:

$$\text{MAD}(d) = \text{mediana}(|d - \text{mediana}(d)|)$$

A mediana é **imune a outliers**: se 5% dos valores de  $d$  são gigantescos (correspondendo a bordas reais), a mediana simplesmente os ignora — eles vão parar nas pontas da lista ordenada, e ela só olha o meio.

Para uma distribuição gaussiana pura, MAD e  $\sigma$  se relacionam por  $\text{MAD} = 0,6745\sigma$ . Combinando com o fator  $\sqrt{2}$  da diferença:

$$\hat{\sigma}_{\text{MAD}} = \frac{\text{MAD}(d)}{0,6745 \cdot \sqrt{2}} \approx \frac{\text{MAD}(d)}{0,9539} \quad (3)$$

Os números 0,6745 e 0,9539 parecem mágicos mas têm origem clara: 0,6745 é a razão entre MAD e  $\sigma$  para a distribuição normal padrão (tabelado em qualquer livro de estatística), e  $\sqrt{2}$  vem do mesmo raciocínio da diferença de gaussianas.

### Atividade C.3

Implemente e compare:

```
1 def estimar_sigma_mad(img):
2     d = np.diff(img, axis=1).ravel()
3     mad = np.median(np.abs(d - np.median(d)))
4     return mad / 0.9539
5
6 print(f"sigma verdadeiro: "
7       f"{sigma_verdadeiro:.4f}")
8 print(f"sigma (diff): {estimar_sigma_diff(g):.4f}")
9 print(f"sigma (mad): {estimar_sigma_mad(g):.4f}")
```

```

7     f"{sigma_verdadeiro:.4f}")
8     print(f"diff (inteira): "
9           f"{estimar_sigma_diff(g):.4f}")
10    print(f"MAD (inteira): "
11          f"{estimar_sigma_mad(g):.4f}")
12    print(f"diff (recorte pinos): "
13          f"{estimar_sigma_diff(recorte_pinos):.4f}")
14    print(f"MAD (recorte pinos): "
15          f"{estimar_sigma_mad(recorte_pinos):.4f}")

```

**Observação:** o MAD recupera o  $\sigma$  verdadeiro mesmo no recorte com muitos pinos. É essa robustez que o torna o método padrão na prática.

## Parte D — Teste de estresse

Vamos quantificar a comparação variando o nível de ruído:

```

1 import pandas as pd # opcional
2
3 niveis = [0.02, 0.08, 0.20] # baixo,
4                               # Gonzalez,
5                               # forte
6 resultados = []
7 for s in niveis:
8     ruído = rng.normal(0, s, f.shape)
9     g = np.clip(f + ruído, 0, 1)
10    resultados.append({
11        'sigma verdadeiro': s,
12        'global (std)': g.std(),
13        'diferencas':     estimar_sigma_diff(g),
14        'MAD':            estimar_sigma_mad(g),
15    })
16
17 df = pd.DataFrame(resultados)
18 print(df.to_string(index=False))

```

O que esperamos:

| $\sigma$ verd. | global         | diff           | MAD            |
|----------------|----------------|----------------|----------------|
| 0,02           | $\approx 0,26$ | $\approx 0,07$ | $\approx 0,03$ |
| 0,08 (G.)      | $\approx 0,26$ | $\approx 0,10$ | $\approx 0,08$ |
| 0,20           | $\approx 0,29$ | $\approx 0,19$ | $\approx 0,17$ |

**Observe:** o erro absoluto do estimador global é sempre da ordem de  $\sigma_f \approx 0,26$ , independente do ruído real. Ele **não vê o ruído**, só vê a estrutura da imagem. Já diff e MAD acompanham o ruído verdadeiro de perto.

## Atenção ao detalhe: o efeito do clip

Você deve ter notado uma curiosidade na linha de  $\sigma = 0,20$ : o MAD recupera  $\approx 0,17$ , e não 0,20. Isso **não é falha do estimador** — é uma consequência do `np.clip(f + ruído, 0, 1)` que aplicamos para garantir que a imagem fique no intervalo válido.

A imagem da placa tem cerca de 6% de pixels já saturados em zero (o fundo preto da radiografia) e algumas regiões claras próximas de 1. Com ruído forte, parte significativa das amostras de  $\eta$  tenta empurrar esses pixels para fora do intervalo  $[0, 1]$  — e o `clip` os trava nas bordas. Resultado: o ruído **efetivamente presente** na imagem  $g$  entregue ao estimador tem desvio menor que o  $\sigma$  que pedimos ao gerador. Em números:

| $\sigma$ pedido | $\sigma$ após clip | % clipados |
|-----------------|--------------------|------------|
| 0,05            | 0,048              | 7%         |
| 0,10            | 0,094              | 10%        |
| 0,20            | 0,180              | 15%        |
| 0,30            | 0,252              | 23%        |

O MAD não está errando — está acertando o  $\sigma$  que **de fato existe** na imagem ruidosa após o `clip`. Em sensores reais isso é exatamente o que acontece: pixels que estouram o limite de saturação travam, e o ruído “visível” na imagem é menor que o ruído “analogico” antes da saturação. O estimador está dando a resposta certa para a pergunta certa.

## Parte E — Voltando ao mundo real

Agora você tem em mãos exatamente a ferramenta que o ISP de um celular precisa: um estimador automático e robusto de  $\sigma$ , sem nunca ter visto a imagem limpa. Em hardware dedicado, isso é implementado em uma única passagem pela imagem — duas linhas de C ou de Verilog — e roda a cada frame, em tempo real, antes do filtro gaussiano.

Num pipeline real o ISP precisa decidir, frame a frame, se aplica um kernel  $5 \times 5$  ou  $21 \times 21$  (e portanto se gasta 10 ou 42 multiplicações por pixel via separabilidade). Essa decisão depende do  $\sigma$  estimado: ruído fraco  $\rightarrow$  kernel pequeno  $\rightarrow$  menos borrramento; ruído forte  $\rightarrow$  kernel grande  $\rightarrow$  mais suavização. Sem a estimação automática, o filtro seria ou genérico demais (sempre o mesmo  $k$ ) ou impossível de configurar.

Os estimadores que você implementou hoje (diferenças e MAD) são, em essência, o que está rodando dentro do silício da câmera do seu celular toda vez que você tira uma foto à noite.

*Você pode fazer essa atividade prática no computador de escolha. Dúvidas? Pergunte. Prepare uma apresentação estilo PPTX bem didática sobre seus experimentos, tabela completa da Parte D, capturas dos resultados das atividades B, C.1, C.2 e C.3 e suas observações. O prazo é de duas semanas como temos feito.*